SOC L1 LAB · INC-006

Change Setup Guide — INC-006

A legitimate change that looks exactly like persistence, on purpose

INCIDENT	INC-006 — Authorized change triage
REPORT DATE	2026-11-01
CLASSIFICATION	Lab exercise — sanitized for publication
ENVIRONMENT	Wazuh 4.14.7 · Debian 12 · QEMU/KVM
SEVERITY	None (authorized change)

Independent portfolio lab. No production systems, client data, or real credentials.

Contents

1. Authorization and safety
2. The change record template
4. Deploy
5. Triage it
6. Rollback
7. Variations
8. Why this is realistic
9. Legal and ethical notes

This guide describes how to stage a **legitimate** maintenance change that trips the cron/systemd persistence rules, then close it with evidence. It is the benign counterpart to INC-004.

Run it only on the isolated lab VM.

---

1. Authorization and safety

Rule	Detail
Scope	Only the `soc-victim` VM
Change	Synthetic, with a synthetic approver
Payload	Log cleanup and `/usr/bin/true`; no network, no account changes
Reversibility	Rollback is part of the change record
Publishing	The change ID and hashes are lab data

The point is to practise the **closure**, not to test an attack.

---

2. The change record template

Write this before deploying. The hashes must come from the exact files you will deploy.

````
# CHG-YYYY-MMDD-NN — <short title>

| Field | Value |
|---|---|
| Status | Approved |
| Requester | <name> |
| Approver | <name> |
| Window | <UTC start–end> |
| Purpose | <one line> |
| Risk | Low / Medium / High |

## Files

| Path | SHA-256 |
|---|---|
| /etc/cron.d/<name> | <hash> |
| /etc/systemd/system/<name>.service | <hash> |
| /etc/systemd/system/<name>.timer | <hash> |

## Rollback

```bash
sudo systemctl disable --now <name>.timer
````
````

```
sudo rm -f /etc/systemd/system/<name>.service /etc/systemd/system/<name>.timer
sudo systemctl daemon-reload
sudo rm -f /etc/cron.d/<name>
```

````
A change record without hashes is not enough for the integrity check.

---

## 3. The files

### 3.1 Cron

```cron
# Nightly cleanup of rotated logs older than 7 days
15 2 * * * root /usr/bin/find /var/log -type f -name '*.gz' -mtime +7 -delete
````

3.2 Service

```
[Unit]
Description=Logrotate health check

[Service]
Type=oneshot
ExecStart=/usr/bin/true
```

3.3 Timer

```
[Unit]
Description=Daily logrotate health check

[Timer]
OnCalendar=daily
Persistent=true
Unit=logrotate-check.service

[Install]
WantedBy=timers.target
```

The service is deliberately a no-op. The cron job only removes old compressed logs.

---

4. Deploy

```
sudo cp /tmp/log-maintenance /etc/cron.d/log-maintenance
sudo cp /tmp/logrotate-check.service /etc/systemd/system/logrotate-check.service
sudo cp /tmp/logrotate-check.timer /etc/systemd/system/logrotate-check.timer
sudo chmod 644 /etc/cron.d/log-maintenance
```
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now logrotate-check.timer
```

Expected alerts:

Rule	Path
100129	`/etc/cron.d/log-maintenance`
100130	`/etc/systemd/system/logrotate-check.service`
100130	`/etc/systemd/system/logrotate-check.timer`
100130	`timers.target.wants/logrotate-check.timer`

---

5. Triage it

1. Read `~/changes/CHG-YYYY-MMDD-NN.md`.
2. Compare the three hashes.
3. Read the cron file and `systemctl cat` both units.
4. Check `systemctl list-timers`.
5. Check auth/FIM/process for follow-on anomalies.
6. Close as **authorized change**, not a false positive.

If any hash or content check fails, stop and escalate.

---

6. Rollback

Use the rollback block in the change record. Record the rollback time and reason in the same file; do not remove a change silently.

---

7. Variations

7.1 Unapproved change

Deploy the same files **without** a change record. The alerts are identical. The correct outcome is escalation, not closure. This is the control test: the detection is not the decision.

7.2 Hash mismatch

Deploy a file, then edit it after recording the hash. The alert still fires; the integrity check fails. Escalate. A hash mismatch is the strongest signal that the deployed artifact is not the approved one.

7.3 Package-installed cron

Install a package that ships `/etc/cron.d/<name>` (the image already has `php` and `e2scrub_all`). The alert is legitimate and should be matched to the package transaction, not treated as an incident.

7.4 Enable-only

Run `systemctl enable` on an existing unit. 100130 fires on the `*.wants` symlink. This is the lowest-signal variant; allow-list the known unit or keep it in the change-aware queue.

---

8. Why this is realistic

- **Most persistence alerts are changes.** The analyst's job is to separate approved changes from unapproved ones, quickly.
- **The files are identical in both cases.** A benign cron file and a malicious one have the same FIM event.
- **The evidence is external to the alert.** The change record and hashes decide.
- **The closure is a skill.** Writing "no incident" with evidence is harder than escalating.

---

9. Legal and ethical notes

- This is a legitimate change on a lab host, not an attack.
- Do not use a synthetic change record to justify a real unauthorized change.
- Keep the change ID and hashes in the lab record.